

ALE 課程 1：為什麼要用 ALE/IDoc？環境查證

Lecture

你們已經學過三種系統整合手段——**REST** ([src/ABAP_Training_REST/](#)，輕量、單次請求/回應)、**RAP** ([src/ABAP_Training_RAP/](#)，OData Service，適合被外部前端或系統即時查詢/操作)、**BAPI** (Interface 課程，RFC-enabled，適合被明確呼叫)。這三種有一個共同前提：**呼叫方知道要主動去問**。但企業整合裡有大量情境是相反的——**發生方** (例如某工廠異動了一筆物料主檔、某公司過了一張發票) 不知道、也不該知道「誰在乎這件事」，只想「把這件事廣播出去，讓在乎的系統自己接手」。**ALE / IDoc** 就是為了這種「廣播式、非同步、系統對系統」的整合情境設計的，是 SAP 最古老、但至今仍是企業對企業 (B2B/EDI)、集團內部多系統同步最主力的技術之一。

ALE 是框架，IDoc 是載體——不要混為一談

- **ALE (Application Link Enabling)**：分散式流程整合的框架，負責決定「哪個系統要跟哪個系統同步什麼資料、什麼時候觸發、要不要過濾」。核心設定是三件事：
 1. **Distribution Model (BD64)**：定義哪個訊息類型 (Message Type) 要從系統 A 分送到系統 B——這是「誰跟誰」的宣告
 2. **Partner Profile (WE20)**：定義每個夥伴系統的通訊細節——這是「怎麼送/怎麼收」的設定 (本課程反覆強調的 Inbound/Outbound 判斷原則就在這裡，見下方)
 3. **Change Pointers (BD61/BD50/BD52)**：主資料異動時自動觸發分送的機制——這是「什麼時候」的其中一種答案 (另一種答案是單據存檔觸發，見下方 Master Data vs Transaction Data)
- **IDoc (Intermediate Document)**：ALE 選定的標準資料格式/訊息容器。一個 IDoc 就是：
 - **Control Record (表頭)**：誰發的、發給誰、什麼訊息類型、什麼時候
 - **Data Records (實際資料)**：按 **Segment (欄位群組)** 結構化，一個 IDoc 可以有 multiple Segment、每個 Segment 可以重複多次 (例如一張訂單的表頭 Segment 一次、明細 Segment 重複 N 次)
 - **Status Record**：這個 IDoc 目前處理到哪個狀態 (03 = 已送出、53 = 對方已處理成功、51 = 對方處理失敗.....)
 - IDoc 的「型別」有三層：**Message Type** (業務語意，如 **MATMAS** = 物料主檔、**INVOIC** = 發票) → **Basic Type** (技術結構版本，如 **MATMAS05**) → 可選的 **Extension** (客製化擴充，見 ale06)

用比喻來說：**ALE 是郵局的物流調度系統** (決定信要從哪裡寄到哪裡、走哪條路線)，**IDoc 是信封 + 信紙的標準格式** (實際裝著內容被寄送的東西)。IDoc 底層走 **tRFC (Transactional RFC)** 或 **EDI 子系統 (WE21 設定的 Port)** 傳輸，確保就算網路斷線也不會漏送或重送。

Master Data 與 Transaction Data：兩種完全不同的觸發機制

ALE / IDoc 用來同步的資料分兩大類，**觸發的機制天生不一樣**，這是本課程刻意分開處理 (ale04 vs. ale05/07) 的原因：

Master Data (主資料)		Transaction Data (交易資料)
例	物料主檔 (MATMAS)、客戶主檔 (DEBMAS)、供應	銷售訂單 (ORDERS)、發票 (INVOIC)、交
子	應商主檔 (CREMAS)、成本中心 (COSMAS)	貨單 (DESADV)

	Master Data (主資料)	Transaction Data (交易資料)
觸發機制	Change Pointer ：主資料異動時系統自動記錄 Change Document， BD52 開啟該訊息類型要追蹤哪些欄位， BD61 全域啟用，標準排程報表 RBDMIDOC 定期把 Change Pointer 打包成 IDoc 送出	Output Determination (NAST 條件技術，掛輸出類型如 RD04) 或 手動/Enhancement 觸發 (本課程 en02/en04 教過的 BTE / Enhancement 掛勾技巧)
性質	框架自動化 ——開發工作量大，重點在 Customizing 設定	事件驅動 ——通常需要自訂程式邏輯，不管是用標準 NAST 還是手寫 Enhancement
本課程對應	ale04	ale05 / ale07 / ale09 / ale10

為什麼這跟「不改標準物件」的專案鐵律有關

跟 Enhancement 課程一樣，ALE / IDoc 的自訂擴充 (Segment 擴充、自訂 Inbound/Outbound Function Module) 全部是「在標準物件旁邊掛一個獨立的擴充」，不需要、也不應該去改標準的 IDoc 結構或標準的 Function Module 本體——這是這整套機制從設計上就支援客製化、又不破壞標準的另一個具體案例。

✅ 已查證的工具支援範圍 (2026-08-24 用 ADT discovery + quickSearch + TADIR 反查實測，非猜測)

IDoc / ALE 三個核心物件 (Message Type / Basic Type / Segment Type) 的 ADT 支援程度三者完全不同，而且結果跟原本的直覺猜測有落差，出乎意料的地方特別要記下來：

物件	交易碼	TADIR 物件型別	ADT 支援
Message Type (如 MATMAS)	WE81	⚠️ 完全沒有 TADIR 個別登記 (SELECT * FROM TADIR WHERE OBJ_NAME = 'MATMAS' 是 0 筆)	無 API——比 GUI-only 物件更徹底，連 TADIR 記錄都沒有。Message Type 的中文/英文說明文字是存在 EDMSG (本系統實測 2327 筆) 這類純資料表裡，不是被追蹤的 Repository 物件，語意上更接近「Customizing 資料」而非「開發物件」
Basic Type / IDoc Type (如 MATMAS05)	WE30	✅ 有 TADIR 登記，物件型別代碼是 IDOC (本系統查到 MATMAS05 屬於套件 MGV)	無 API——discovery 全文搜尋 idoc/segment/ale/distribution 關鍵字皆 0 命中，quickSearch 對 IDOC 型別物件也是 0 結果 (跟 T-code / CMOD 不同，這裡連查得到的唯讀 metadata stub 都沒有，是「有 TADIR 但完全沒有任何 ADT 曝光」的案例)
Segment Type (如 E1MARAM)	WE31	🏆 意外發現： TADIR 物件型別代碼其實是 TABL (本系統查到	✅ 完全可以走一般 DDIC Structure API ! quickSearch 直接找得到 (回傳 TABL/DS)， sap_get_source (objectType=STRU) 已實測成功完整讀出 E1MARAM 的原始碼 (define structure e1maram { ... })

物件	交易碼	TADIR 物件型別	ADT 支援
		E1MARAM 屬於套件 IDOCLOGISTICS)	所有欄位清一色 <code>abap.char(n)</code> ——IDoc Segment 的慣例是不管底層資料元素原生型別是什麼，一律用字元型別存放，因為 IDoc 本質是純文字格式的 EDI 訊息)

這對 **ale06** (自訂 IDoc 擴充) 有直接的教學意義：Segment 的欄位結構本身理論上可以走一般 DDIC Structure 的建立流程 (`sap_create_object/sap_set_source` , `objectType=STRU`) 由 Claude 自動建立與驗證，只有「把這個 Structure 登記成一個真正的 Segment」(版本管理、Release 狀態、掛進 Basic Type) 這個動作需要 **WE31/WE30** GUI 手動完成——這是本課程繼 Enhancement 課程「en04 的 ENHOXHH 骨架 GUI-only、內容可以 ADT 讀寫」之後，又一個「物件登記動作 GUI-only、但底層資料結構可以 ADT 自動化」的案例，實際可行性會在 ale06 出題時進一步驗證 (是否真的能用 `sap_create_object` 建出一個能被 **WE31** 辨識的新 Segment) 。

(2026-09-21 補充細化，ale06 出題時發現)：Segment 其實是兩層 DDIC Structure——**E1MARAM** (Segment Type，欄位全是 `abap.char(n)`，對應 IDoc 資料記錄的扁平文字) 與 **E2MARAM** (Segment Definition，欄位引用真正的 Data Element)，中繼資料在 **EDISDEF** / **EDSAPPL**。ADT 可以讀這兩層 Structure，但不能靠自建 Structure 來註冊 Segment (**EDISDEF** / **EDSAPPL** 只有 **WE31** 會寫入)，所以 ale06 的 Segment 建立仍走 GUI，詳見 ale06。

BD64 / WE20 / WE21 / BD61 / BD50 / BD52 / WE19 / WE02 / WE05 / BD87 十個交易碼全部確認 TADIR 物件型別是 **TRAN**——延續 `.claude/rules/sap-adt-mcp.md` 第 12 節已記載的結論，T-code 完全沒有 ADT API，這幾個 ALE 環境設定與監控交易碼一律走 GUI-only、Claude 寫操作指引的教學模式 (ale02/ale03)。

附帶確認：**EDIDC** (IDoc Control Record 表) 本系統已有 **334 筆真實資料**——代表系統上已經有實際的 IDoc 收發歷史，之後可以用 `datapreview/freestyle` 查詢這些既有資料，找標準情境當教材範例，不用每次都憑空生資料。

學習目標

- 能用一句話講清楚 ALE (框架) 跟 IDoc (載體) 的關係，不會把兩者混為一談
- 能講出 IDoc 三層結構 (Control/Data/Status Record) 跟三層型別 (Message Type/Basic Type/Extension)
- 能分辨 Master Data (Change Pointer 驅動) 跟 Transaction Data (Output Determination / 手動觸發) 兩種分送機制的差異，並判斷一個新情境該歸類到哪一種
- 知道 ALE / IDoc 跟你們已經學過的 REST / RAP / BAPI 在「呼叫模式」上的根本差異 (廣播式非同步 vs. 主動查詢式)
- 知道 Message Type / Basic Type / Segment Type 三者 ADT 支援程度上完全不同，尤其記住「Segment Type 本質是一張 DDIC Structure」這個意外但重要的事實

事前準備

已用 `sap-adt` / `sap-adt-home` MCP (on-premise 系統，client 130) 於 2026-08-24 完成查證，方法與結果見 Lecture「 已查證的工具支援範圍」一節：

-  ADT discovery 全文搜尋 `idoc` / `segment` / `partner` / `message` / `ale` / `distribution` 關鍵字，除了無關的 Message Class (**SE91**) 與 Enterprise Services Proxy Message Type (PI/PO 用，跟 IDoc 無關)

之外，沒有任何 IDoc 專屬 collection

- ☒ TADIR 反查確認：Message Type (**MATMAS**) 查無 TADIR 記錄；Basic Type (**MATMAS05**) TADIR 型別是 **IDOC** (有登記但無 ADT 曝光) ；Segment Type (**E1MARAM**) TADIR 型別其實是 **TABL** (一般 DDIC Structure，`sap_get_source(objectType=STRU)` 已實測成功讀出完整原始碼)
- ☒ **BD64 / WE20 / WE21 / BD61 / BD50 / BD52 / WE19 / WE02 / WE05 / BD87** 十個交易碼皆確認 TADIR 型別為 **TRAN**，無 ADT API
- ☒ **EDIDC** 確認已有 334 筆真實 IDoc Control Record 資料，**EDMSG** 有 2327 筆 Message Type 說明文字資料

題目需求

1. 完成 **Master Data vs Transaction Data** 對照表 (可參考 Lecture 內容整理，重點是理解每一格的理由，不是照抄)：資料類型 / 觸發機制 / 誰啟動流程 / 本課程對應題目。
2. 情境判斷 (針對下面三個情境，先判斷該用哪種分送機制，再說明理由)：
 - 情境一：集團內兩家子公司要保持客戶主檔一致，其中一家異動了客戶的付款條件，需要另一家系統自動跟著更新
 - 情境二：公司 A 對公司 B (集團內關聯公司) 開立了一張 Invoice Verification，需要公司 B 自動記一筆對應的應收帳款
 - 情境三：一批物料的安全庫存量在系統 A 被整批調整，需要同步給負責另一個廠區採購的系統 B
3. **ALE vs. 你們已學過技術的比較**：用一張表比較 ALE/IDoc 跟 REST (`src/ABAP_Training_REST/`) 在「觸發方式」、「呼叫方是否需要主動查詢」、「適合的整合情境」三個面向的差異。
4. 重現查證 + 解讀：用 `sap-adt/sap-adt-home` MCP 或直接 `curl datapreview/freestyle` 查 `SELECT OBJECT, OBJ_NAME, DEVCLASS FROM TADIR WHERE OBJ_NAME = 'E1EDK01'` (發票類 IDoc 的表頭 Segment)，確認也是 **TABL** 型別；再用 `sap_get_source(objectType=STRU)` 讀出它的原始碼。用一句話解釋：為什麼 Segment Type 會被實作成一張 DDIC Structure，而不是獨立的物件型別？

參考答案 (情境判斷)

- 情境一：**Master Data · Change Pointer 機制**——客戶主檔異動是典型的主資料同步情境，開 **BD52** 追蹤付款條件欄位、**BD61** 全域啟用即可，不需要自訂觸發程式。
- 情境二：**Transaction Data · 手動/Enhancement 觸發**——這是單一事件 (MIRO 過帳) 驅動的自訂情境，**MIRO** 沒有像銷售單據那樣天生的 Output Determination 機制可掛，需要用 Enhancement/BTE 掛勾點主動組 IDoc 送出 (ale09 期末案例)。
- 情境三：**Master Data · Change Pointer 機制**——物料主檔的安全庫存量欄位異動，同樣走 Change Pointer (**MATMAS** 訊息類型)，只要在 **BD52** 確認該欄位有被追蹤即可。
- 題目 4 參考答案：**E1EDK01** 同樣是 **TABL** 型別 (發票 IDoc **INVOIC01/INVOIC02** 的表頭 Segment)。Segment 之所以被實作成一張 DDIC Structure，是因為它本質上就只是「一組欄位定義」——不需要方法、不需要商業邏輯，跟一般 Structure 的用途 (描述資料形狀) 完全一致；SAP 沒有必要為此另外發明一套獨立的物件型別與維護工具，直接重用 DDIC Structure 的基礎設施 (含版本管理、Data Element 引用) 最有效率，**WE31** 只是在這張 Structure 之上疊加一層「這是一個 IDoc Segment」的中繼資料 (版本號、Release 狀態、掛在哪個 Basic Type 底下)。

思考題

1. 如果 Master Data 的 Change Pointer 機制這麼方便 (幾乎不用寫程式)，為什麼 Transaction Data 不能比照辦理，也做一套「異動就自動記錄、排程打包送出」的機制？ (提示：想想 Master Data 異動的頻率/即時性要求，跟 Transaction Data [例如一張發票] 的即時性要求有什麼本質差異)

2. ALE 是「廣播式」整合——發送方不需要知道接收方會怎麼處理。這種設計對「發送方」有什麼好處？對「除錯 / 追蹤問題」又會帶來什麼挑戰？（提示：對照你們在 REST 課程學到的「呼叫方直接拿到回應」模式來想）

答案

不需要新建任何 SAP 物件。Master Data vs Transaction Data 對照表、情境判斷、ALE vs. REST 比較表見本題內文的參考答案。系統查證已於 **2026-08-24** 用 **sap-adt/sap-adt-home MCP (on-premise , client 130)** 完成：Message Type (**WE81**) 無 TADIR 登記、無 ADT API；Basic Type (**WE30** , TADIR 型別 **IDOC**) 有登記但無 ADT 曝光；**Segment Type** (**WE31** , TADIR 型別其實是 **TABL**) 可以完全用一般 **DDIC Structure API** 讀取 (已用 **sap_get_source(objectType=STRU)** 對 **E1MARAM / E1EDK01** 實測成功)；**BD64/WE20/WE21/BD61/BD50/BD52** 等十個交易碼確認皆為 **TRAN** 型別、無 ADT API。完整查證方法與結果見 Lecture 「 已查證的工具支援範圍」一節，已同步更新 README 環境限制段落。